



Git & GitHub

완전정복

버전관리 시스템의 이해와 실습 — 개념부터 협업 명령어까지

총 9개 강의 · Git/GitHub 기초부터 실전 프로젝트까지 한 번에

전체 커리큘럼

01

Git/GitHub란?

개념·필요성·기본 용어·설치 및 가입

02

저장소 만들어보기

Linux/vi 기초·저장소 생성·Git 원리

03

기본 명령어 알아보기

clone·pull·revert·branch·checkout

04

GitHub 기본기능 알아보기

README·Markdown·Issue·Wiki·Gist

05

Git 중급 명령어

merge, conflict, amend, stash

06

버전 관리 & Git Flow

Tag, git flow 전략

07

GitHub 중급기능

fork·Pull Request

08

Git 심화 명령어

cherry-pick·merge 방식·rebase

09

실전 프로젝트

쇼핑몰 프로젝트로 Git·GitHub 실전 연습



Git/GitHub 란?

버전관리의 개념과 필요성을 이해하고,
Git 설치와 GitHub 회원가입까지 진행합니다.

1강에서 배울 내용



Git의 개념



GitHub의 개념



분산버전 관리 툴의 필요성



Git/GitHub 기본 용어 살펴보기



Git 설치해보기



GitHub 회원가입해보기

Git이란?



2005

리누스 토르발스 개발

- 컴퓨터 파일의 변경사항을 추적하고, 여러 사용자 간 작업을 조율하기 위한 분산 버전 관리 시스템
- 소프트웨어 개발의 소스 코드 관리에 주로 사용되지만, 어떤 파일 집합의 변경 이력 추적에도 활용 가능
- 2005년, 리눅스 커널 개발을 위해 리누스 토르발스와 커널 개발자들이 함께 개발

GitHub란?



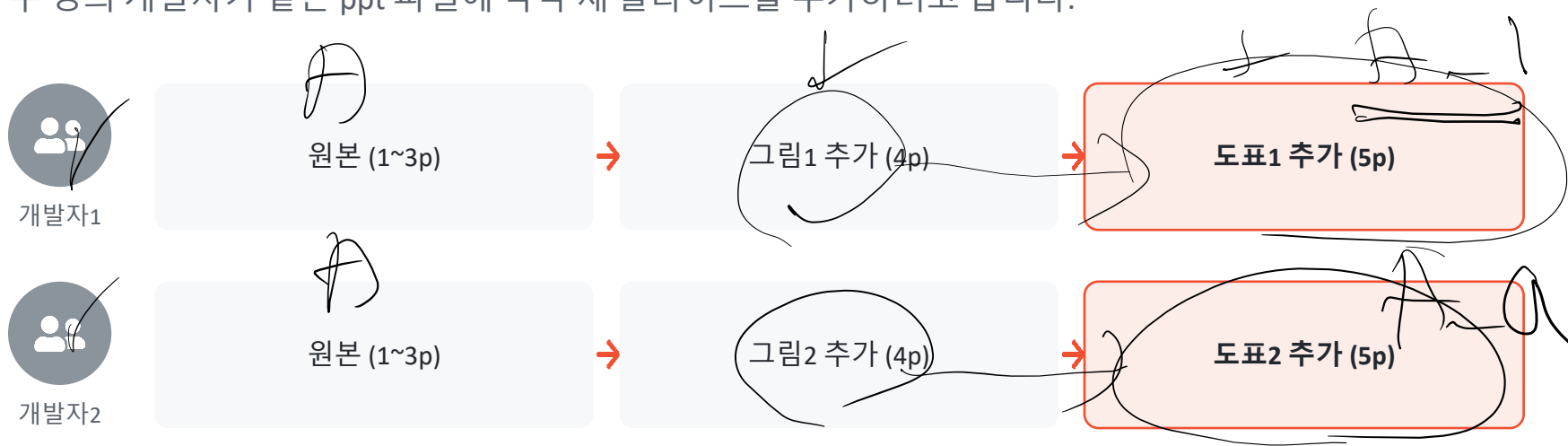
2008

설립 · 2018 MS 인수

- Git을 사용하는 프로젝트를 지원하는 웹호스팅 서비스로, 가장 인기 있는 Git 호스팅 사이트
- CLI 기반인 Git과 달리 화려한 GUI를 제공하며, Gist·Wiki 등 협업 기능을 저장소마다 운영
- 톰 프레스턴워너 등이 2008년 공동 설립, 2018년 6월 마이크로소프트가 75억 달러에 인수

왜 분산 버전관리가 필요할까?

두 명의 개발자가 같은 ppt 파일에 각각 새 슬라이드를 추가하려고 합니다.

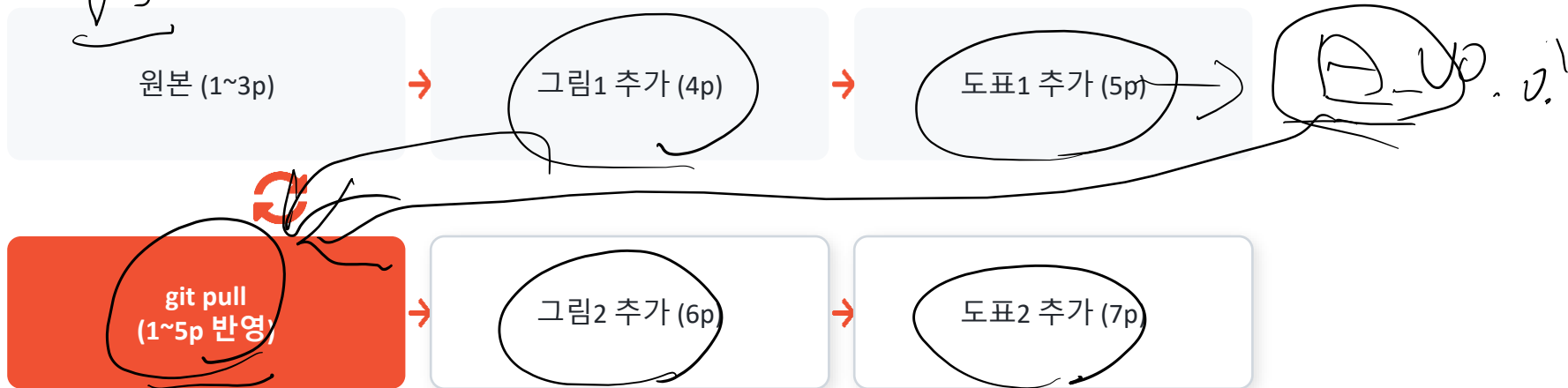


Git 없이는 서로 독립된 2개의 ppt 파일이 생성되어, 누구의 작업도 반영되지 않습니다.

WHY GIT · 해결 방법

Git으로 해결하기

개발자2가 git을 사용해 개발자1의 작업 내역을 pull 받은 뒤 이어서 작업하면, 하나의 완성된 파일만 남습니다.



결과: 개발자1과 개발자2의 작업이 모두 담긴 1개의 최종 파일 완성!

Git의 장점



변경 취소 가능

버그 있는 코드도 언제든지 이전 버전으로 되돌릴 수 있습니다.



"히스토리" 관리

모든 변경 내역이 남아 작업 이력을 한눈에 파악할 수 있습니다.



동시 개발 가능

☆
Branch를 통해 여러 갈래로 동시에 개발을 진행할 수 있습니다.

BENEFITS

GitHub의 장점



편리한 GUI

CLI인 Git과 달리 편리한 그래픽 인터페이스를 제공합니다.



Issue · Wiki 협업

Issue와 Wiki 기능으로 효율적인 협업을 가능합니다.



Pull Request

Pull Request로 코드 리뷰와 공동 개발이 가능합니다.

핵심 용어 정리

Git

분산 버전 관리 툴

GitHub

Git을 더 유용하게 쓰도록 돕는 웹호스팅 서비스

commit

한 번의 변경 내역을 반영하는 단위 (예: 버튼 추가 commit)

Repository

Git으로 관리하는 저장소 (예: 쇼핑몰 소스코드 저장소)

GUI

마우스로 컴퓨터와 상호작용하는 그래픽 환경

CLI

키보드로 컴퓨터와 상호작용하는 터미널 환경

nacl

실습: Git 설치 & GitHub 가입



Git 설치하기

- Mac : 기본적으로 git이 설치되어 있습니다.
- Windows / Linux : 아래 사이트에서 설치할 수 있습니다.

`git-scm.com`
`/downloads`



GitHub 회원가입

- github.com 접속 후 Sign up 진행
- 이메일로 전송된 Verification 확인을 누르면 가입이 완료됩니다.

`github.com`

1강 정리



Git이 무엇인지 살펴보았습니다.



GitHub이 무엇인지 살펴보았습니다.



분산 버전관리 툴의 필요성을 학습했습니다.



Git과 GitHub 각각의 장점을 살펴보았습니다.



Git을 설치하고 GitHub에 회원가입해보았습니다.



저장소 만들어보기

기본 Linux/vi 명령어를 익히고,
나만의 Git 저장소와 GitHub 원격 저장소를 만들어봅니다.

2강에서 배울 내용



기본 Linux 명령어들을 살펴봅니다.



기본 vi 사용법을 살펴봅니다.



Git 저장소 만들어보기



첫번째 commit 만들어보기



Git의 원리 – Working Directory·Staging·Repository



.gitignore로 불필요한 파일 제외하기



GitHub 저장소 만들어보기

CLI와 기본 Linux 명령어



Git Bash는 Windows에서 Bash Shell 환경을 구성해주는 프로그램이며, Git은 이 터미널(CLI) 환경에서 명령어로 사용됩니다.

pwd

print working directory. 현재 작업 경로를 반환합니다.

cd

다른 경로로 현재 작업 경로를 변경합니다.

ls

현재 경로의 모든 파일 목록을 반환합니다.

mkdir

현재 경로에 새로운 디렉토리를 생성합니다.

vi 편집기 핵심 명령어

vi는 유닉스 환경에서 가장 많이 쓰이는 화면 단위(visual) 문서 편집기입니다.

모드 전환

i

insert 모드 (커서 위치에 입력)

a

append 모드 (커서 한 칸 뒤 입력)

o

open line 모드 (다음 줄에 입력)

esc

normal 모드로 전환

저장 & 종료

:w

저장 (write)

:q

종료 (quit)

:wq

저장 후 종료

:wq!

강제 저장 후 종료

커서 이동

h

왼쪽으로 이동

j

아래로 이동

k

위로 이동

l

오른쪽으로 이동

Git 전역 설정하기



git config

가입한 GitHub 계정과 동일한 이름.이메일로 사용자 정보를 설정합니다.

```
git config --global user.name "test name"  
git config --global user.email testmail@gmail.com
```

Git 저장소 생성하기



git init

원하는 폴더에서 실행하면 숨김 폴더 .git이 생성되며 Git 저장소가 만들어집니다.

```
git init
```

첫번째 Commit 만들기



git add

파일 변경 내역을 추가

```
git add testfile.txt
```



git status

현재 작업 내역 확인

```
git status
```



git commit

변경 내역을 저장소에 반영

```
git commit -m "텍스트 파일 추가"
```



git log

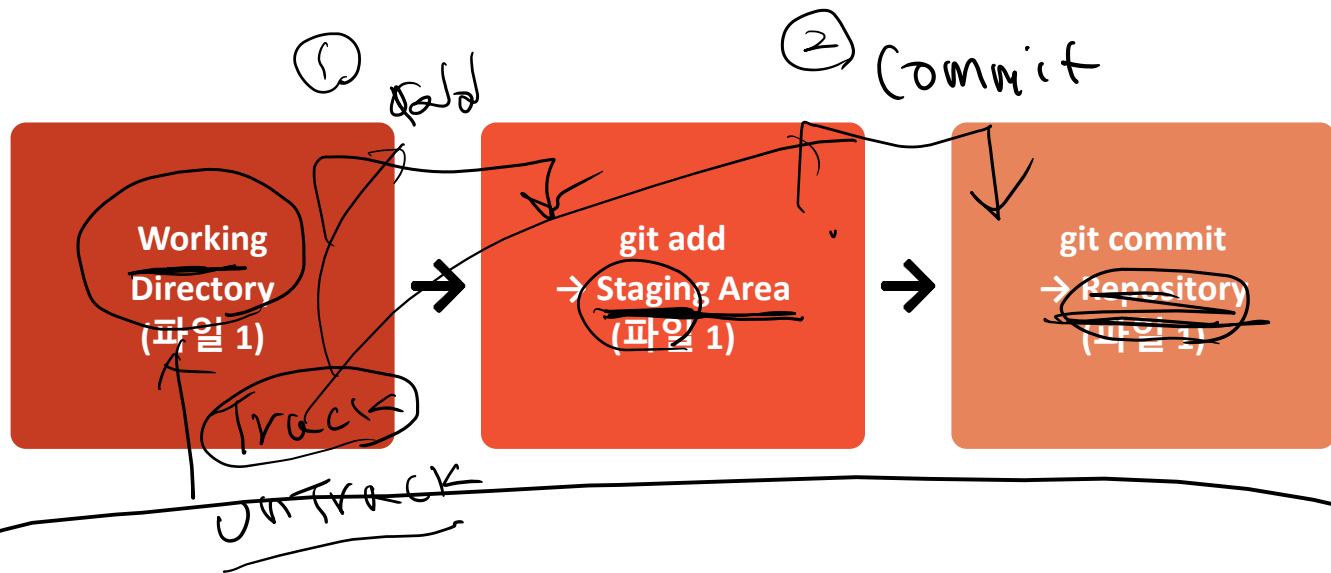
작업한 commit 내역 확인

```
git log
```

Git의 원리 -

Working Directory · Staging · Repository

Git에서 파일을 관리하는 세 영역의 흐름을 살펴봅시다.



로컬 원격 저장소 연결

로컬 저장소

변경1

변경2



git push

로컬 원격 저장소 (bare)

```
cd D:/TEST  
Git init --bare  
PwD D:/TEST
```

```
git remote add origin D:/TEST
```

```
git remote -v
```

원격 연결 확인

```
git push origin master
```

변경 내역을 원격 저장소에 반영

REMOTE ADD · PUSH

GitHub 원격 저장소 연결

로컬 저장소



git push

원격 저장소 (GitHub)



```
git remote add origin https://github.com/GitHub아이디/저장소이름.git
```

```
git remote -v
```

원격 연결 확인

```
git push origin master
```

변경 내역을 원격 저장소에 반영

.gitignore 파일

- 변경사항을 추적하고 싶지 않은 파일이나 확장자 등은 별도의 .gitignore 파일을 통해 관리할 수 있습니다.
- .gitignore 파일을 통해 불필요한 파일들을 추적대상에서 제거할 수 있습니다.
- 실습: .gitignore 파일을 직접 작성해봅시다.

2강 정리



기본 Linux 명령어(pwd, cd, ls, mkdir)를 살펴보았습니다.



기본 vi 사용법을 살펴보았습니다.



git init · git config로 저장소와 사용자 정보를 설정했습니다.



git add · status · commit · log로 첫 commit을 만들어보았습니다.



Git의 원리(Working Directory·Staging·Repository)를 이해했습니다.



.gitignore로 불필요한 파일을 관리에서 제외하는 방법을 살펴보았습니다.



git remote add · push로 GitHub 원격 저장소와 연동했습니다.



Git 기본명령어 알아보기

clone · pull · revert · branch · checkout

협업에 필요한 핵심 명령어를 실습합니다.

3장에서 배울 내용



`git clone`



`git pull`



`git revert`
`git reset`



`git branch`



`git checkout`

원격 저장소 가져오기 · 반영하기



git clone

GitHub 원격 저장소의 내용을 로컬에 받아옵니다.(최초 1회)

```
git clone https://github.com/GitHub아이디/저장소이름.git
```



git pull

원격 저장소의 변경 내역을 로컬 저장소에 반영합니다.(최신 상태로 갱신)

```
git pull origin master
```

원격: 변경1,2



로컬: 변경1 반영

잘못된 Commit 되돌리기



- git revert 명령어로 잘못 commit한 내역을 이전으로 되돌릴 수 있습니다.
- 되돌리고 싶은 commit의 hash는 git log 명령어로 확인합니다.

```
git revert "되돌리고 싶은 commit hash"
```

```
git log
```

Commit 완전히 되돌리기



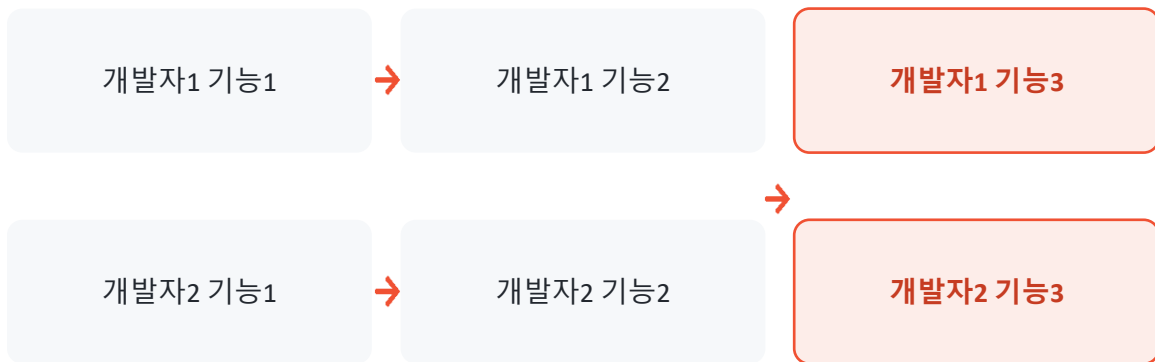
- git reset 명령어로 지정한 commit 시점으로 완전히 되돌릴 수 있습니다.
- git revert와 달리 그 이후의 commit 이력 자체가 삭제되므로 사용에 주의가 필요합니다.
- 되돌리고 싶은 commit의 hash는 git log 명령어로 확인합니다.

```
git reset --hard "되돌리고 싶은 commit hash"
```

```
git log
```

Branch가 필요한 이유

하나의 소스코드를 여러 명이 동시에 개발하면 충돌이 발생할 수 있어, 저장소의 가지(Branch)를 나눠 독립적으로 작업합니다.



Branch 없이는 두 작업이 한 흐름 위에서 충돌합니다 — 각자의 가지(Branch)로 나뉘야 합니다.

Branch 관리 명령어

```
git branch developer2
```

새로운 branch 생성

```
git checkout developer2
```

작업중인 branch 전환 (Head 이동)

```
git branch
```

현재 branch 목록 확인

```
git branch -D developer2
```

branch 삭제

```
git push origin :developer2
```

삭제 내역을 원격 저장소에 반영

3강 정리



git clone으로 원격저장소 내용을 로컬에 받아오는 방법을 살펴보았습니다.



git pull로 원격 저장소의 변경 내역을 로컬에 반영하는 방법을 살펴보았습니다.



git revert로 잘못 commit한 내역을 되돌리는 방법을 살펴보았습니다.



git branch / checkout으로 branch를 생성·전환·삭제하는 방법을 살펴보았습니다.



GitHub 기본기능 알아보기

README·Markdown부터 Issue, Wiki, Gist,
Star·Follow·Watch까지 살펴본 뒤, 다음 시간엔 실제 협업의 위기를 만나봅니다.

4강에서 배울 내용



README.md



Markdown 문법



Issue 생성·Close



Wiki



GitHub commit 확인 · Milestone 관리



Star · Follow · Watch

CONCEPT

README.md

- README.md는 GitHub 저장소에 대한 기본 설명으로 보여지는 파일입니다.
- GitHub에서 .md 파일은 Markdown 문법에 따라 화면에 렌더링됩니다.
- 실습: README.md 파일을 직접 생성하고 Markdown을 작성해봅시다.

Markdown이란?

- 마크다운(Markdown)은 일반 텍스트 기반의 경량 마크업 언어로, 일반 마크업 언어보다 문법이 쉽고 간단합니다.
- HTML·리치 텍스트(RTF) 등으로 쉽게 변환되어 README 파일이나 온라인 게시물 등에 널리 사용됩니다.

Markdown 문법



글자 크기

#, ##, ### 등으로 글자 크기(제목 수준)를 조정할 수 있습니다.

```
# 제목1    ## 제목2    ### 제목3
```



리스트

들여쓰기와 - 기호로 리스트를 만들 수 있습니다.

```
- 아이템 1
  - 서브 아이템
```



굵은 글씨

****~**** 로 굵은 글씨를 만들 수 있습니다.

```
**굵은 텍스트**
```

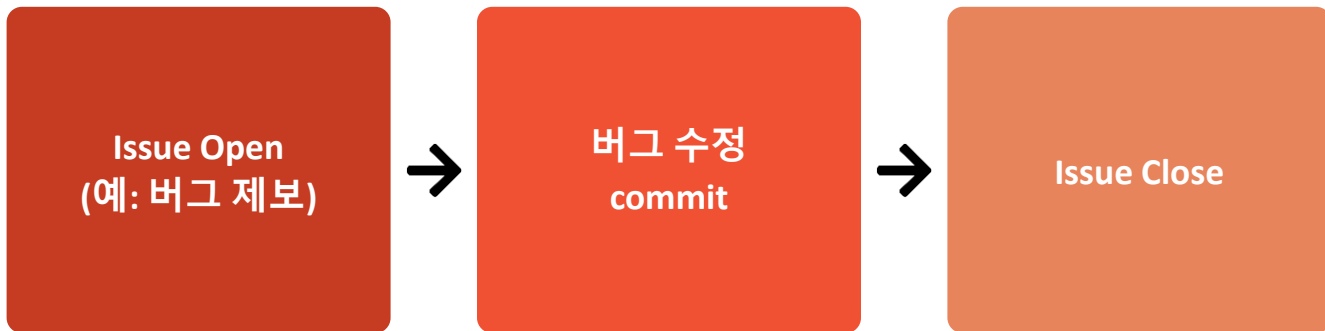
CONCEPT

Issue

- GitHub Repository의 Issue 탭에서 Issue를 생성할 수 있습니다.
- 버그를 제보하거나 다른 작업자들과 공유할 내용을 Open 할 수 있습니다.
- 각 Issue별로 tag를 설정할 수 있으며, 관리자는 Issue를 Close 상태로 변경할 수 있습니다.

Issue 처리 프로세스

Issue의 처리 프로세스는 다음과 같습니다.

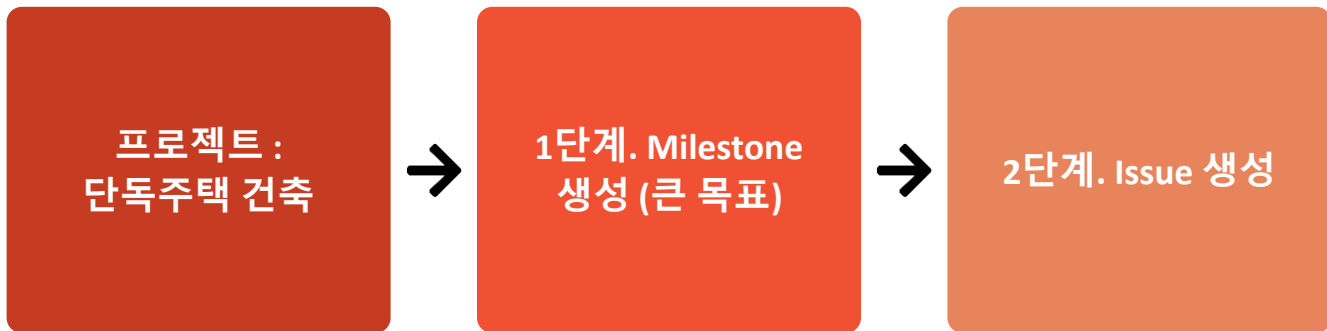


GitHub Milestones로 프로젝트 관리하기

- GitHub Milestones에서 새로운 Milestone을 생성합니다.
- Milestone에 특정 일자까지 진행할 목표를 설정할 수 있습니다.
- Issue를 생성할 때 Issue별로 Milestone을 할당할 수 있습니다.

Issue & Milestones 처리 프로세스

목표(Milestone)를 먼저 만들고, 그 목표에 속하는 작업(Issue)을 연결하는 것이 가장 일반적인 사용 순서



Milestone	목표	수행해야 할 작업 (Issue) 등록	Issue	담당자
기초공사	기초공사 완료		터파기	김철수
골조공사	건물 뼈대 완성		철근 배근	이영희
설비공사	전기·수도 등 설비 완료		거푸집 설치	박민수
인테리어	내부 마감 완료			
준공	최종 검사 및 입주			

CONCEPT

GitHub Wiki

- 위키(Wiki)는 불특정 다수가 협업을 통해 직접 내용과 구조를 수정할 수 있는 웹사이트를 일컫습니다.
- GitHub의 Wiki를 통해 프로젝트에 대한 설명을 작성할 수 있습니다.
- 실습: GitHub Wiki를 직접 작성해봅시다.

CONCEPT

GitHub Gist

- GitHub은 저장소 관리 외에도 하나의 파일에 대한 변경 이력을 손쉽게 관리하고 공유할 수 있는 GitHub Gist 서비스를 제공합니다.
- Github gist는 짧은 코드, 메모 등을 기록 또는 공유 목적으로 사용할 수 있는 무료 서비스
- 실습: GitHub Gist를 직접 생성해봅시다.

<https://gist.github.com/>

Star · Follow · Watch



Star

마음에 드는 저장소에 Star를 눌러서 관심 저장소로 설정할 수 있습니다.



Follow

마음에 드는 유저를 follow해서 활동내역을 확인할 수 있습니다.



Watch

관심 저장소에 Watch를 눌러서 변경내역을 추적할 수 있습니다.

GitHub commit 내역 확인하기

- commits 버튼을 누르면 어떤 유저가 언제 어떤 내역을 수정했는지 한눈에 볼 수 있습니다.

4강 정리



README.md 파일에 대해 살펴보았습니다.



Markdown 문법에 대해 살펴보았습니다.



Issue를 생성하고 Close하는 방법을 살펴보았습니다.



Wiki 작성 방법과 GitHub Gist에 대해 살펴보았습니다.



GitHub Star·Follow·Watch 기능을 살펴보았습니다.



commit 내역 확인과 Milestones 기능을 살펴보았습니다.



Git 중급명령어 알아보기

– merge, conflict, amend, stash

저장소 만들 때 배운 원리를 떠올리며, 이제 두 사람이 함께 작업할 때 생기는 merge와 conflict를 직접 겪어봅니다. (amend·stash도 함께)

5강에서 배울 내용



git merge



conflict 해결



git commit --amend



git stash

CONCEPT

두 개의 branch 내용을 합치기 - git merge

- 독립적으로 개발되던 2개의 branch에 있는 내용을 merge를 이용해서 합칠 수 있습니다.
- 먼저 합칠 대상이 되는 branch(예: master)로 이동한 뒤, 합치고 싶은 branch를 지정해줍니다.
- 실습: git merge 명령어로 2개의 branch 내용을 합치는 과정을 실습해봅시다.

git merge 명령어



git checkout

합칠 대상이 되는 master branch로 먼저 이동합니다.

```
git checkout master
```



git merge

merge 명령어로 합치고 싶은 branch를 지정해줍니다.

```
git merge developer2
```

CONFLICT

두 branch를 합치는 과정에서 발생한 conflict

- 2개의 branch에서 동일한 파일을 수정한 뒤 commit하고 이를 merge하면 conflict(충돌)이 발생할 수 있습니다.
- 예를 들어 master branch와 developer2 branch가 같은 README.md 파일에 서로 다른 내용을 작성한 뒤 merge하면, 어떤 내용을 반영해야 할지 몰라 conflict가 발생합니다.
- conflict이 발생하면 수동으로 해결해서 새로운 commit을 만들어 주어야 합니다.

conflict이 발생한 파일 상황



충돌 마커

conflict가 발생한 파일에는 아래와 같은 마커가 표시됩니다.

```
<<<<<< HEAD
master
=====
developer2
>>>>>> developer2
```

이전 commit 내역 수정하기 - amend



git commit --amend

작업 중 실수로 commit에 빠트린 내용이 있을 때, 새 commit 대신 amend로 이전 commit에 내용을 추가할 수 있습니다.

```
git commit --amend
```

변경사항을 잠시 keep 해두기 - git stash

- 특정 branch에서 작업 중에 다른 branch에서 급한 작업을 요청받는 경우가 생길 수 있습니다.
- 예: feature branch에서 새 기능을 개발 중인데 master branch에 급한 버그를 수정해야 하는 상황.
- 이때 작업 내역을 commit하기 애매하다면, 변경사항을 잠시 보관해두는 git stash 명령어를 사용할 수 있습니다.

git stash 명령어



git stash save

작업 내역을 stash에 저장합니다.

```
git stash save
```



git stash list

stash에 저장된 내용을 확인합니다.

```
git stash list
```



git stash pop

stash에 저장해 두었던 내용을 다시 가져옵니다.

```
git stash pop
```

5강 정리



git merge 명령어로 여러 개의 branch를 합치는 과정을 살펴보았습니다.



merge 과정에서 conflict가 생길 수 있는 상황과 해결 방법을 살펴보았습니다.



amend 명령어로 이전 commit 내역을 수정하는 방법을 살펴보았습니다.



git stash 명령어로 작업 내역을 임시 저장하고 다시 불러오는 방법을 살펴보았습니다.



소프트웨어 버전 관리 & Tag · branch 관리 전략 - Git Flow

버전관리 규칙과 tag 명령어를 익히고,

머지와 충돌을 경험했으니, 이제 Git Flow 브랜치 전략으로 팀의 규칙을 정리합니다.

6강에서 배울 내용



소프트웨어 버전관리



Tag 생성 및 삭제



branch 생성 전략 – Git Flow



소프트웨어 출시 과정

소프트웨어 버전 관리

v2.4.1



Major

대규모 기능 추가 등 Major 업데이트가 이루어질 시에 변경



Minor

세부 기능 추가 등 Minor한 업데이트가 이루어질 시에 변경



Patch

버그 패치 등의 사소한 변경이 이루어질 시에 변경

버전명을 기록·확인·반영하기



git tag -a

현재 소스코드 상태에 대한 기록을 tag로 생성합니다.

```
git tag -a v2.4.1
```



git tag -l

생성된 tag 목록을 확인합니다.

```
git tag -l
```



git push (tag)

생성된 tag를 원격 저장소에 반영합니다.

```
git push origin v0.0.1
```

Tag 삭제하기



git tag -d

생성한 tag를 로컬에서 삭제합니다.

```
git tag -d v0.0.1
```



git push (삭제)

원격 저장소에 올라간 tag는 tag명 앞에 :를 붙여 삭제합니다.

```
git push origin :v0.0.1
```

Branch 생성 전략 - Git Flow

master

제품으로 출시될 수 있는 브랜치

develop

다음 출시 버전을 개발하는 브랜치

feature

기능을 개발하는 브랜치

release

이번 출시 버전을 준비하는 브랜치

hotfix

출시 버전에서 발생한 버그를 수정하는 브랜치

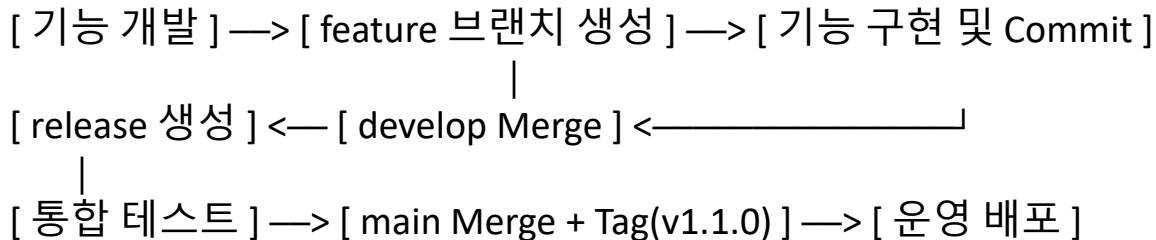
Git Flow 개발 흐름

- master 브랜치에서 시작해 동일한 develop 브랜치를 생성하고, 개발자들은 develop 브랜치에서 개발을 진행합니다.
- 새 기능이 필요하면 develop에서 feature 브랜치를 생성해 구현한 뒤, 검토를 거쳐 다시 develop에 합칩니다(Merge).
- 모든 기능이 완료되면 develop을 release 브랜치로 만들어 QA를 진행하고, 완료되면 master·develop에 반영 후 tag를 생성해 배포합니다.
- 배포 후 발견되지 않은 버그가 있으면 hotfix 브랜치를 만들어 긴급 수정하고 태그를 생성해 바로 배포합니다.

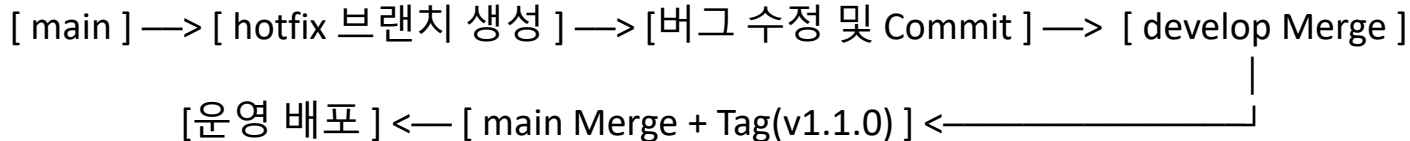
Branch 생성 전략 - Git Flow



일반 개발 프로세스

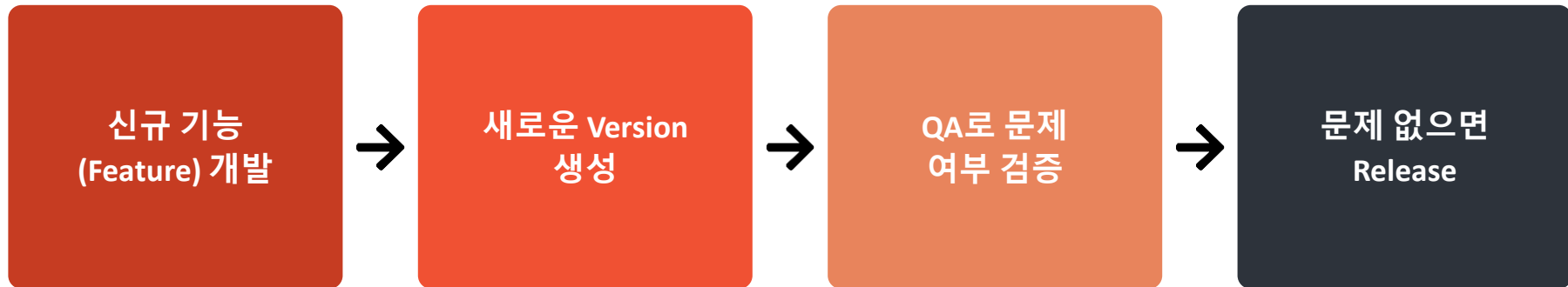


긴급 개발 프로세스



소프트웨어 출시 과정

일반적으로 소프트웨어는 다음과 같은 과정을 거쳐서 출시합니다.



6강 정리



소프트웨어 버전 관리 방법(Major·Minor·Patch)에 대해 살펴보았습니다.



git tag 명령어로 tag를 생성·삭제하는 방법을 살펴보았습니다.



소프트웨어 출시 과정에 대해 살펴보았습니다.



branch 생성 전략인 Git Flow 전략에 대해 살펴보았습니다.



GitHub 중급기능 알아보기

– fork, pull request

Git Flow로 팀의 규칙을 익혔으니, 이제 팀 밖 사람들과도 협업해봅니다.
원본 저장소를 복제하는 fork와 협업의 핵심인 Pull Request를 배웁니다.

7강에서 배울 내용



GitHub fork



Pull Request

CONCEPT

GitHub fork - 원본 저장소를 복제해서 나만의 작업 공간 만들기

- 오픈소스 등 대규모 프로젝트에서 작업자 모두가 자신의 branch를 만든다면 엄청나게 많은 branch가 생성됩니다.
- 이때 branch 대신 GitHub fork를 이용하면 원본 저장소와 독립적인 나만의 복사된 저장소를 만들 수 있습니다.
- 실습: GitHub fork를 직접 실습해봅시다.

CONCEPT

GitHub Pull Request – Fork에서 작업한 내역을 메인 Repository에 Merge 요청

- Fork한 저장소에서 작업한 내역을 메인 Repository에 Merge해달라고 요청할 때 Pull Request를 사용할 수 있습니다.
- 메인 저장소는 Pull Request만을 통해 Merge를 허용하는 정책을 사용하면, 코드 리뷰를 거쳐 안전하게 협업할 수 있습니다.
- 실습: GitHub Pull Request를 직접 실습해봅시다.

7강 정리



GitHub fork를 이용해서 나만의 저장소를 복제하는 방법을 살펴보았습니다.



GitHub Pull Request를 이용해서 작업 내역에 대한 Merge를 요청하는 방법을 살펴보았습니다.



Git 심화 명령어 알아보기

– cherry-pick, rebase

특정 commit만 반영하는 cherry-pick과 merge의 두 가지 방식,
그리고 history를 깔끔하게 유지하는 rebase를 살펴봅니다.

8강에서 배울 내용



git cherry-pick



git merge – fast-forward, 3-way merge



git rebase

특정 commit만 다른 branch에 추가하기



git cherry-pick

다른 branch에 있는 특정 commit만 반영하고 싶을 때 사용합니다. (예: 2.0 branch의 긴급 버그 수정만 master에 반영)

```
git cherry-pick "반영하고 싶은 commit hash"
```



git log --branches

모든 branch의 commit 로그를 확인할 때 사용합니다.

```
git log --branches
```

CONCEPT

Git Merge의 2가지 방식

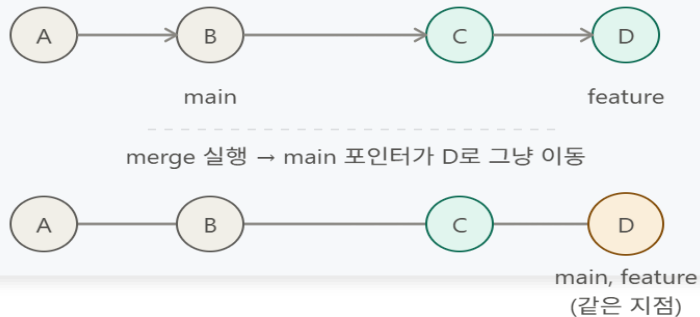
- 2개의 branch를 합치는 명령어에는 merge와 rebase, 두 가지 방식이 있습니다.
- merge 알고리즘에는 다시 두 가지 방식이 있습니다: Fast-forward merge와 3-way merge.

Fast-forward merge

- branch 생성 이후 master에 추가 commit이 없을 경우, 별도의 merge commit 없이 branch가 master 위치까지 빨리 감기 형태로 이동하며 merge가 진행됩니다.
- master branch에 developer2 branch를 merge하는 대표적인 경우입니다.
- 실습: Fast-forward Merge를 직접 실습해봅시다.

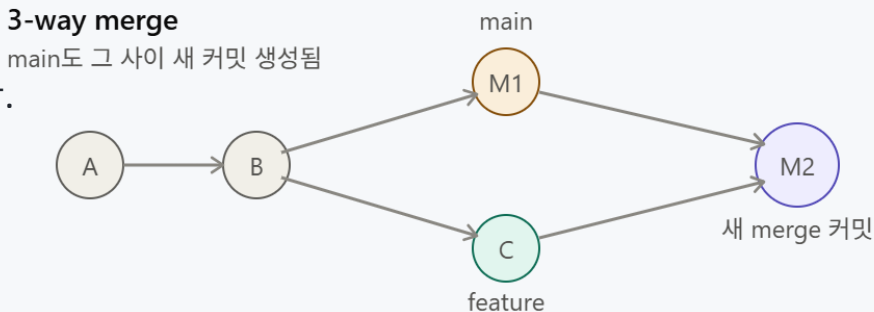
Fast-forward merge

main이 그 사이 변화 없음



3-way merge

- branch 생성 이후 master에도 추가 commit이 있을 경우, 두 커밋을 모두 반영하는 merge commit이 별도로 생성된 뒤 merge가 진행됩니다.
- 대규모 소프트웨어 개발에서 이런 merge commit이 지속적으로 생기면 나중에 history 관리가 어려워질 수 있습니다.
- 실습: 3-way merge를 직접 실습해봅시다.



Git rebase - base를 바꿔서 합치는 방법



git rebase

base를 다른 branch의 최신 commit으로 바꿔주어, 병합 과정에서 별도의 merge commit이 생성되지 않게 해줍니다.

```
git rebase <rebase 대상이 되는 branch>
```

	Merge	Rebase
결과 히스토리	갈래가 남음 (다이아몬드 모양)	일직선으로 깔끔하게 정리됨
새 커밋	merge 커밋 하나 추가	기존 커밋들이 새 커밋으로 다시 생성됨 (해시 값이 바뀜)
원본 커밋	그대로 보존	사라지고 복제본으로 대체
언제 쓰나	협업 히스토리를 있는 그대로 남기고 싶을 때	히스토리를 깔끔하게 정리하고 싶을 때

8강 정리



git cherry-pick 명령어를 이용해서 다른 branch에 있는 특정 commit만 반영하는 방법을 살펴보았습니다.



merge의 2가지 방식인 fast-forward merge와 3-way merge에 대해 살펴보았습니다.



git rebase 명령어의 개념과 실습을 진행해보았습니다.



실전 프로젝트로 연습하는 Git/GitHub

2명의 개발자가 쇼핑몰 웹사이트를 만드는 상황을 가정해서,
지금까지 배운 Git과 GitHub를 실전처럼 연습해봅니다.

9강에서 배울 내용



프로젝트 시나리오 소개



Git Flow 브랜치 전략 복습



업무 분담 및 Milestone·Issue 관리



개발 → merge → tag → wiki

2명의 개발자가 쇼핑몰 웹사이트를 만드는 상황을 가정

- 두 명의 개발자(developer1, developer2)가 쇼핑몰 웹사이트를 만드는 상황을 가정해서 지금까지 배운 Git과 GitHub를 실습해봅니다.
- 각자 독립적인 branch에서 기능을 개발한 뒤, master branch로 merge하는 실전 협업 흐름을 경험해봅니다.

Branch 생성 전략 - Git Flow

- 효율적인 협업과 소프트웨어 개발을 위한 다양한 branch 생성 전략들이 있습니다.
- 이 중에서 가장 유명한 Vincent Driessen의 Git Flow 전략 (master·develop·feature·release·hotfix)을 이번 프로젝트에도 적용해봅니다.

업무 분담

각 개발자별 업무 할당 및 작업 프로세스

쇼핑몰 웹사이트 개발을 위해서 개발자1과 개발자2가 각각의 기능을 나눠서 개발하기로 하였습니다. Git/GitHub을 이용한 실제 업무 진행과정을 체험해봅시다.

developer1

메인 페이지

회원 가입 페이지

구매후기 페이지

developer2

상품 소개 페이지

결제 페이지

배송안내 페이지

shoppingmall 개발을 위한 Git/GitHub 실습

- 각 기능을 개발하는 상황을 실습을 통해서 Git/GitHub을 이용한 실제 업무 진행과정을 체험해봅시다.
- 실습: shoppingmall 저장소를 생성하고 README.md로 첫 commit을 만들어봅시다.

예제 코드

저장소 생성 & 첫 Commit

```
$ mkdir shoppingmall && cd shoppingmall  
$ git init  
$ echo "# Shopping Mall Project" > README.md  
$ git add README.md  
$ git commit -m "Initial commit: README.md 추가"  
$ git remote add origin https://github.com/team/shoppingmall.git  
$ git push -u origin master
```

기능별 Milestone 생성

1. 개발자별 기능을 Milestone으로 생성합니다.
2. Milestone별로 각 개발자가 작업내역을 Issue로 설정하고, 완전히 작업이 끝나면 Issue를 close하는 형태로 관리합니다.

Developer1의 개발 시나리오

1. developer1이라는 branch를 생성합니다.
2. 해당 branch로 checkout합니다.
3. 메인 페이지를 commit 합니다.
4. 회원 가입 페이지를 commit 합니다.
5. 구매후기 페이지를 commit 합니다.

예제 코드

developer1 브랜치 작업 코드

```
$ git checkout -b developer1
```

```
# 메인 페이지 작업
```

```
$ git add main.html && git commit -m "feat: 메인 페이지 추가"
```

```
# 회원 가입 페이지 작업
```

```
$ git add signup.html && git commit -m "feat: 회원 가입 페이지 추가"
```

```
# 구매후기 페이지 작업
```

```
$ git add review.html && git commit -m "feat: 구매후기 페이지 추가"
```

```
$ git push origin developer1
```

Developer2의 개발 시나리오

1. developer2이라는 branch를 생성합니다.
2. 해당 branch로 checkout합니다.
3. 상품 소개 페이지를 commit 합니다.
4. 결제 페이지를 commit 합니다.
5. 배송안내 페이지를 commit 합니다.

예제 코드

developer2 브랜치 작업 코드

```
$ git checkout -b developer2
```

```
# 상품 소개 페이지 작업
```

```
$ git add product.html && git commit -m "feat: 상품 소개 페이지 추가"
```

```
# 결제 페이지 작업
```

```
$ git add payment.html && git commit -m "feat: 결제 페이지 추가"
```

```
# 배송안내 페이지 작업
```

```
$ git add delivery.html && git commit -m "feat: 배송안내 페이지 추가"
```

```
$ git push origin developer2
```

MERGE

master branch에 두 개발자의 개발내역을 merge

1. master branch에서 develop1 branch를 merge합니다.
2. master branch에서 develop2 branch를 merge합니다.
3. merge 과정에서 conflict이 발생할 경우 수동으로 conflict을 해결해줍니다.

master 브랜치 Merge 코드

```
$ git checkout master
$ git merge developer1
$ git merge developer2
Auto-merging main.html
CONFLICT (content): Merge conflict in main.html
# 충돌 부분을 직접 열어 수동으로 해결한 뒤 저장
$ git add main.html
$ git commit -m "Merge branch 'developer2' - conflict 해결"
$ git push origin master
```

개발내역에 대한 tagging과 wiki 생성

1. Merge가 끝난 branch에 대해서 v0.1.0으로 tag를 생성합니다.
2. 개발내역에 대한 history 관리를 위해서 wiki를 작성합니다.

예제 코드

Tag 생성 & Wiki 작성 코드

```
$ git tag -a v0.1.0 -m "shoppingmall v0.1.0 release"
```

```
$ git push origin v0.1.0
```

```
$ git tag
```

```
v0.1.0
```

```
# GitHub 저장소 > Wiki 탭 > New Page
```

```
# 제목: Home
```

```
# 내용: 프로젝트 개요, 브랜치 전략, 개발 히스토리 기록
```

9강 정리



2명의 개발자가 쇼핑몰 웹사이트를 만드는 프로젝트 시나리오를 실습해보았습니다.



Git Flow 브랜치 전략에 따라 업무를 분담하고 Milestone·Issue로 진행 상황을 관리해보았습니다.



각자 branch에서 기능을 개발하고 commit한 뒤, master branch로 merge하는 과정을 실습해보았습니다.



merge가 끝난 버전에 tag를 생성하고 wiki로 개발 내역을 정리해보았습니다.



깃허브에 이력서 사이트와 블로그 만들기

지킬(Jekyll) 테마를 활용해 나만의 블로그를 만들고,
GitHub Pages로 전 세계에 공개해봅니다.

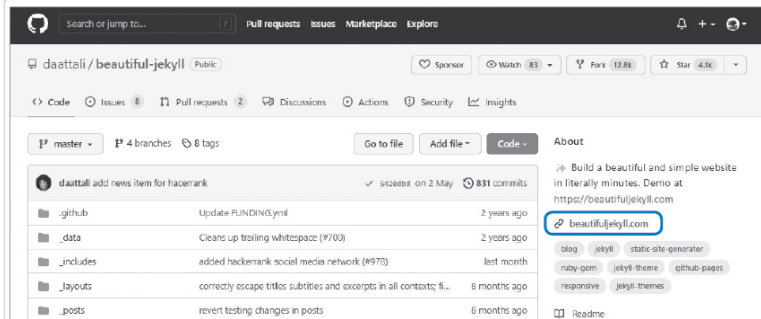
지킬 테마 살펴보기

- 블로그의 목적을 먼저 생각하고 테마를 골라야 함 (자주 쓰는 개인 블로그 vs 이력서·업무용 사이트)
- 최신 글이 앞에 보이고 온라인에서 수정하기 편리한 테마가 좋음
- 유료·무료 테마가 모두 있으며, 무료 테마만으로도 개인 블로그 운영은 충분
- 검색 사이트에서 'Jekyll Theme'로 검색하면 테마 목록과 Demo 링크 확인 가능
- 지킬 테마는 대부분 GitHub 저장소 형태로 배포됨

참고 사이트 github.com/topics/jekyll-theme · jekyllthemes.org · jekyll-themes.com

지킬 테마 선택하기

- GitHub 검색창에 'jekyll' 검색 → 관련 테마 저장소 목록 확인
- 즐겨찾기(★) 수로 테마의 인기도 확인 가능
- 테마 클릭 → README의 Demo 링크로 실제 적용 화면 미리보기
- 지킬 테마는 대부분 저장소 자체가 소스와 사용법을 함께 제공



블로그 만들고 환경설정하기 (1)

- 1 'jekyll-now' 저장소로 이동해 README의 사용법과 데모 링크를 확인합니다.
- 2 마음에 들면 Fork 버튼을 클릭해 내 계정으로 저장소를 복제합니다.
- 3 저장소 이름은 반드시 '깃허브아이디.github.io' 형식으로 지정합니다. (예: jjlaon → jjlaon.github.io)
- 4 Create fork를 클릭하면 master 브랜치 그대로 내 계정에 복제됩니다.

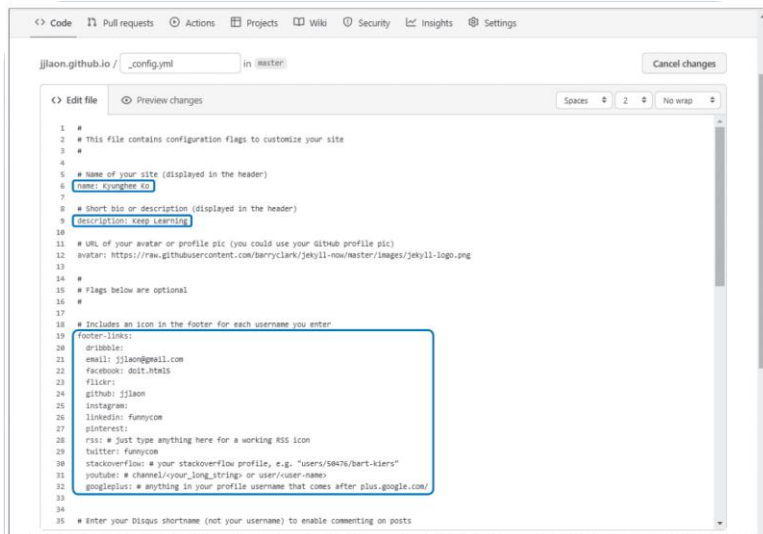
블로그 만들고 환경설정하기 (2)

- 복제한 지킬 테마를 내 블로그에서 쓰려면 테마 환경을 나에게 맞게 설정해야 함
- 저장소 파일 목록에서 `_config.yml` 파일을 클릭
- 연필(✎) 아이콘을 클릭하면 파일을 웹에서 바로 수정할 수 있는 편집 화면으로 이동

파일명 `_config.yml` — 지킬 테마의 전체 환경설정을 담당하는 파일

블로그 만들고 환경설정하기 (3)

- 항목마다 달린 주석을 참고해 자신에게 맞게 값을 수정
- name, description, footer links의 url은 반드시 수정
- 간단한 커밋 메시지와 함께 Commit changes 클릭
- 새 탭에서 블로그 주소 접속 → 수정한 내용이 반영됐는지 확인



블로그에 포스트 작성하기 (1)

- `_posts` 디렉터리의 기본 포스트 `2014-3-3-Hello-World.md`를 열어 형식을 확인
- `---` 부터 `---` 까지(Front Matter)는 모든 포스트에 필수 · `layout: post`는 수정 금지, `title`만 수정
- 새 글은 [Add file] → [Create new file], 파일명은 '날짜-제목.md' 형식 (예: `2022-06-10-first.md`)
- 복사해둔 Front Matter를 맨 위에 붙여넣고 `title`을 바꾼 뒤, 마크다운 문법으로 본문 작성

블로그에 포스트 작성하기 (2)

- Preview 탭에서 결과 미리보기, 수정이 필요하면 Edit new file 탭에서 소스 수정
- 화면 아래에 커밋 메시지를 입력하고 Commit new file 클릭
- 커밋 후 _posts 디렉터리에 새 포스트 파일이 생성된 것을 확인
- 이후 새 글을 쓸 때도 같은 방법으로 _posts에 파일을 추가하면 됨 (반영까지 약간의 시간 소요)

지킬 테마 페이지에 나만의 .md 파일 추가하기

커밋 충돌 시나리오

- 마크다운 문법으로 나만의 .md 파일을 만들어 블로그 페이지에 추가해보기
- _posts/ 폴더에 새 파일을 만들고 로컬과 GitHub 양쪽에서 같은 파일을 수정해 충돌을 재현
- 충돌 발생 시 표시 기호를 직접 확인하고 원하는 내용만 남겨 해결

확인 주소 `https://yyouri.github.io/`

HANDS-ON · 실습 8



GitHub Actions로

자동 뉴스 크롤링 웹페이지 만들기

개발 지식이 없어도, 이 가이드를 그대로 따라 하면
매일 자동으로 IT 뉴스를 모아 보여주는 내 웹페이지를 완성할 수 있습니다.

총 11단계 · 예상 소요시간 60~90분

이 실습에서 무엇을 만드나요?



매일 정해진 시간에

GitHub Actions가 자동으로 실행돼요



파이썬 코드가

IT 뉴스 사이트에서 인기 뉴스 10개를 가져와요



결과가 자동으로

내 GitHub 저장소에 저장되고 웹페이지로 공개돼요



즉, 사람이 손대지 않아도 컴퓨터가 알아서 매일 새로고침 해주는 웹페이지를 만드는 것입니다.

BEFORE YOU START

시작 전 준비물 체크리스트



GitHub 계정

github.com 에서 무료로 가입

github.com/signup



Git 설치

컴퓨터에 Git 프로그램 설치

git-scm.com/downloads



Python 설치

3.10 이상 버전 설치

python.org/downloads



VS Code 설치

코드를 작성할 편집기

code.visualstudio.com

이미 설치되어 있다면 이 단계는 건너뛰어도 됩니다.

전체 진행 순서 한눈에 보기

1 GitHub에 새 저장소 만들기

2 내 컴퓨터로 저장소 가져오기 (clone)

3 파이썬 개발 환경 준비하기

4 필요한 라이브러리 목록 작성하기

5 크롤링 코드 작성하기

6 로컬에서 실행해 확인하기

7 완성한 코드 GitHub에 올리기 (push)

8 GitHub Pages로 웹사이트 켜기

9 워크플로우 파일 만들기 — 실행 조건

10 워크플로우 파일 만들기 — 실행 단계

11 자동 실행 결과 확인하기

GitHub에 새 저장소 만들기

- 웹 브라우저에서 `github.com/new` 접속 (로그인 상태여야 합니다)
- Repository name(저장소 이름)에 `python_crawler` 입력
- Public(공개) 선택 — GitHub Pages 무료 공개를 위해 Public이어야 합니다
- “Add a README file” 체크박스 선택
- Create repository 버튼 클릭 → 저장소 생성 완료!



저장소(Repository)란? 내 프로젝트 파일들을 모아 보관하고, 변경 이력을 관리하는 GitHub 상의 폴더입니다

내 컴퓨터로 저장소 가져오기

- 내 컴퓨터에 프로젝트를 저장할 폴더로 이동한 뒤, 그 안에서 터미널(Git Bash) 열기
- 아래 명령어로 방금 만든 저장소를 내 컴퓨터에 복제(clone)합니다

```
git clone https://github.com/YYouri/python\_crawler.git .  
cd python_crawler
```

- VS Code 실행 → File(파일) > Open Folder(폴더 열기) → 방금 만든 python_crawler 폴더 선택
- VS Code 상단 메뉴 Terminal(터미널) > New Terminal 로 터미널을 열어두면 이후 모든 명령어를 여기서 입력할 수 있습니다

파이썬 개발 환경 준비하기

① 파이썬이 설치되어 있는지 확인

```
python --version
```

버전 번호(예: Python 3.11.4)가 나오면 정상 설치된 것입니다.

② 가상환경 만들고 켜기 (이 프로젝트 전용 파이썬 공간)

```
python -m venv venv  
venv\Scripts\activate
```

Windows

```
python3 -m venv venv  
source venv/bin/activate
```

Mac / Linux

터미널 맨 앞에 (venv) 표시가 보이면 가상환경이 켜진 것입니다.

필요한 라이브러리 목록 작성하기

- VS Code 왼쪽 탐색기에서 python_crawler 폴더 위에 마우스 우클릭 → New File
- 파일 이름을 requirements.txt 로 저장하고, 아래 두 줄을 그대로 입력

```
requests  
beautifulsoup4
```

- 터미널에서 아래 명령어로 라이브러리를 한 번에 설치합니다

```
pip install -r requirements.txt
```



requests는 웹페이지를 가져오는 도구, beautifulsoup4는 가져온 페이지에서 원하는 정보를 뽑아내는 도구입니다.

크롤링 코드 작성하기 · `crawl_news.py`

- 같은 방법으로 `crawl_news.py` 파일을 새로 만들고 아래 코드를 그대로 입력합니다

```
import requests

from bs4 import BeautifulSoup
from datetime import datetime
import os

os.makedirs('docs', exist_ok=True)
URL = "https://m.etnews.com/news/hot_content_list.html"
res = requests.get(URL)
soup = BeautifulSoup(res.text, 'html.parser')

section = soup.select_one("section.textthumb")
top = section.select('ul li strong a')[:10]

now = datetime.now().strftime('%Y-%m-%d')
with open('docs/index.md', 'w', encoding='utf-8') as f:
    f.write(f"# {now} 많이 본 뉴스\n\n")
    for i, a in enumerate(top, 1):
        title = a.text.strip()
        link = "https://m.etnews.com"+a['href']
        f.write(f"{i}. [{title}]({link})\n")
```



docs 폴더를 만들어요 (웹페이지로 공개할 폴더)



뉴스 사이트 페이지를 가져와요



가져온 페이지에서 인기 뉴스 10개를 찾아요



제목과 링크를 docs/index.md 파일에 순서대로 적어요

로컬에서 실행해 확인하기

- 터미널에서 아래 명령어로 코드를 실행합니다

```
python crawl_news.py
```

- 왼쪽 탐색기에 docs 폴더와 그 안에 index.md 파일이 새로 생겼는지 확인합니다
- index.md 파일을 열어 오늘 날짜와 뉴스 제목 10개 목록이 보이면 성공입니다



오류가 난다면? → requirements.txt 설치를 안 했거나(4단계 다시 확인), 인터넷 연결을 확인해보세요. 오류 메시지를 그대로 복사해 AI에게 물어보면 빠르게 원인을 알 수 있습니다.

완성한 코드 GitHub에 올리기

- 터미널에서 아래 명령어를 순서대로 입력해 코드를 GitHub 저장소로 올립니다(push)

```
git add .  
git commit -m "크롤링 코드 추가"  
git push origin main
```

`git add .`

변경된 모든 파일을 이번 저장 대상으로 등록

`git commit -m "..."`

지금까지 변경 내용을 하나의 저장 지점으로 기록

`git push origin main`

내 컴퓨터의 기록을 GitHub 저장소로 업로드

GitHub 저장소 페이지를 새로고침해서 파일들이 올라갔는지 확인해보세요.

GitHub Pages로 웹사이트 켜기

- GitHub 저장소 페이지 상단의 Settings(설정) 탭 클릭
- 왼쪽 메뉴에서 Pages 클릭
- Build and deployment > Source에서 "Deploy from a branch" 선택
- Branch를 main, 폴더를 /docs 로 선택한 뒤 Save 클릭
- 잠시 후 상단에 나타나는 초록색 주소(https://아이디.github.io/python_crawler/)가 내 웹페이지 주소입니다



GitHub Pages는 docs 폴더 안의 index.md를 자동으로 웹페이지(HTML)로 변환해서 보여줍니다. 별도의 웹 서버가 필요 없습니다.

워크플로우 파일 만들기 ① 실행 조건

- VS Code 탐색기에서 .github 폴더를 만들고, 그 안에 다시 workflows 폴더를 만듭니다
- workflows 폴더 안에 python-package.yml 파일을 새로 만들고 아래 내용부터 입력합니다
`name: Daily News Crawler`

```
on:
  schedule:
    - cron: '0 21 * * *'      # 매일 06:00 (KST) 자동 실행
  workflow_dispatch:         # Actions 탭 수동 실행 버튼 추가

permissions:
  contents: write            # 저장소에 커밋·푸시할 권한 부여

jobs:
  crawl:
```

`cron`은 실행 시각을 정하는 규칙, `workflow_dispatch`는 수동 실행 버튼을 만들어줍니다.

워크플로우 파일 만들기 ② 실행 단계

- 바로 이어서 실제로 실행할 작업 단계(steps)를 아래처럼 작성합니다

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with: { python-version: '3.11' }
  - run: pip install -r requirements.txt
  - run: python crawl_news.py
  - run: |
      git config --global user.name 'github-actions'
      git config --global user.email 'actions@github.com'
      git add .
      git commit -m "Update news" || echo "변경 없음"
      git push origin HEAD:${{ github.ref_name }}
```

 checkout(코드 가져오기) → setup-python(파이썬 준비) → 설치 → 크롤링 실행 → 결과를 다시 커밋·푸시는 순서입니다.

자동 실행 결과 확인하기

- `git add .` → `git commit -m "자동화 워크플로우 추가"` → `git push origin main` 으로 워크플로우 파일을 올립니다
- GitHub 저장소 상단의 Actions 탭 클릭
- "Daily News Crawler" 워크플로우가 보이면, 오른쪽의 Run workflow 버튼으로 지금 바로 한 번 실행해볼 수 있습니다
- 초록색 체크(✓)가 뜨면 성공! 내 웹페이지 주소에 접속해 뉴스 목록이 갱신되었는지 확인합니다
- 이후로는 cron에 설정한 시간마다 자동으로 실행됩니다



축하합니다! 매일 자동으로 갱신되는 나만의 뉴스 웹페이지가 완성되었습니다.

403 Permission denied (권한 오류)



이런 메시지가 보인다면

```
remote: Permission to 아이디/python_crawler.git denied to github-actions[bot].  
fatal: The requested URL returned error: 403
```

해결 방법

- 저장소 Settings → Actions → General 이동
- "Workflow permissions" 항목에서 Read and write permissions 선택 후 Save
- Actions 탭에서 실패한 워크플로우의 Re-run jobs 클릭

그 밖에 자주 발생하는 문제들



ModuleNotFoundError: No module named 'bs4'

워크플로우 파일에 `pip install -r requirements.txt` 단계가 빠졌는지 확인하세요.



웹페이지가 404 Not Found로 나옴

Settings > Pages에서 Branch/폴더가 main, /docs로 정확히 지정됐는지 확인하세요.



Actions 탭에 워크플로우가 안 보임

.github/workflows/ 폴더 경로와 파일 확장자(.yml)가 정확한지 확인하세요.



크롤링 결과가 비어있음

크롤링 대상 사이트의 구조가 바뀌었을 수 있어요. select 부분의 태그명을 다시 확인하세요.



감사합니다

Git과 GitHub의 기본기를 다졌으니, 이제 직접 저장소를 만들고
commit·branch·push를 자유롭게 다뤄보세요.